Disclaimer: *Dieses OER-Dokument stammt aus dem Projekt proTirone, das über GitHub freie Unterrichtsmaterialien samt Quellcode offeriert. proTirone-Materialien werden unter den Bedingungen der CC-BY-4.0-Lizenz so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

LFCX:sh2go:

1. Allgemeines

- **Scripts** = Programmdateien, die von einem Interpreter zeilenweise abgearbeitet.
  - *Bash-Script* = abgearbeitet von der Shell `bash`
  - *Dash-Script* = abgearbeitet von der Shell `dash`
  - *psh-Script* = abgearbeitet von der Powershell
  - *python-Script* = abgearbeitet vom Pythoninterpreter
- typischer Aufruf
  - `bash bash-script.sh`
  - `dash dash-script.sh`
  - `pwsh pwsh-script.ps1`
  - `python3 python-script.py`

Unter Linux gibt es nativ eine Abkürzung:

- Das Script enthält als Erstes die *Shebang-Zeile*: `#!` gefolgt vom absoluten Pfad zum Interpreter:

```
1 #!/bin/bash
2 # Shebangsymbol '#!' + absoluter Pfad zum Interpreter '/bin/bash'
3
4 echo "Hello World"
```

- Das Script wird mit `chmod 755 bash-script.sh` ausführbar gemacht.
- Dann ist das Script ohne Interpreter aufrufbar.

Unter Windows gibt es die Abkürzung - Stand heute - nativ nicht, sondern nur über Zusätze.

- `bash`, `dash` und `pwsh` können unter Windows 11 und Linux ausgeführt werden.
- Mindestens die `pwsh` hat auf den Systemen aber unterschiedliche Fähigkeiten.

Zweck: Shellskripte machen Befehlsaufrufe wiederholbar.

2. Nutzung der *Powershell 7.5.x*

- *Nutzungsbegründung:* Nur Powershell 7 erlaubt Nutzung des WSL 2

→ **Zur Geschichte:**

Windows

- enthielt ursprünglich nur die Command-Shell (`cmd`):
  - Suche in Windowssuche nach *cmd* oder *command*.
  - Klicke angezeigte *Eingabeaufforderung* an
  - Windows startet die Command-Shell zum Starten einfacher Programme
- Im Jahr 2003 hat Windows (Unix-)Admins eingeladen, um ihre Wünsche an eine bessere Admin-Shell zu erfahren. Ergebnis: So, wie unter UNIX/Linux.
- Daraufhin entstand die Powershell als eine auf den .NET-Framework aufsetzende Shell, auch mit eingebauten Befehlen.
  - Version 1.0 im Jahr 2006
  - Version 2.0 im Jahr 2008
  - Version 3.0 im Jahr 2012
  - Version 4.0 im Jahr 2013
  - Version 5.0 im Jahr 2015 zusammen mit Windows 10 (für ältere in Version 5.1 nachgereicht)
  - Version 6.0 im Jahr 2018 ist eine neue Open-Source-Version [Powershell-Core] (MIT) gehostet auf GitHub, die auch für LNX erhältlich ist.
  - Version 7.+x ist die aktuelle, 5.1 kriegt nur noch updates, 6 wird nicht weiterentwickelt, Basis ist jetzt die Version 7.x plattformübergreifend.

Hinweis: Linux-Shells und Powershell unterscheiden sich trotzdem grundsätzlich:

- PWSH arbeitet .NET basiert. Die meisten Kommandos sind als NET-Funktionalitäten im PWSH-Code implementiert.
- Linux-Shells dagegen stellen nativ die Programmlogik und Variablenauflösung bereit, ansonsten rufen sie externe Tools auf.

Installation

Powershell 7 ersetzt nicht vollständig Powershell 5. Deshalb existiert Powershell 5 auch noch nach der Installation von Powershell 7 und wird oft als defaultmäßig gestartet.

- Ermitteln der eigenen Version:
  - Powershell starten

    - > `$psVersionTable` eingeben

- Ggf. Powershell Version 7.5 installieren:
    - per Windowsstrore ohne Adminrechte:
        * suche darin nach Powershell 7.
        * lasse es installieren
    - per → GitHub-Repository
        * suche `PowerShell-7.5.2-win-x64.exe` unter https://github.com/PowerShell/PowerShell/releases/tag/v7.5.2
        * Lade es runter in Deinen privaten App-Ordner.
        * Lasse dafür Link in Panel / Start anlegen

- Installation verifizieren:
    - Starte Powershell 5:
        * In Appsuche nach *Powershell* (sic!) suchen und starten
        * > `$psVersionTable` eingeben
    - Starte Powershell 7:
        * In Appsuche nach *pwsh* (sic!) suchen und starten oder
        * In gestarteter Powershell-5 `pwsh` aufrufen
        * · > `$psVersionTable` eingeben

Hinweis: Powershell 7 (= pwsh) gibt es als Binaries auch für Linux und Mac. Allerdings: Die Powershell unter Microsoft arbeitet mit .NET-Funktionen, während die und Linux (und Mac?) meistens Tools des Betriebssystems aufrufen. Deshalb ist die `pwsh` streng gesehen nicht Betriebssystem-übergreifend.

ÜBUNG **LF11a:sbj-00.sh2go:01**

☐ Installieren Sie die aktuellste Powershell auf Ihrem Windowsrechner.
☐ Verifizieren Sie ihre Installation

---

3. Nutzung von *bash* / *dash*

- *Nutzungsbegründung*:

  - Shellprogrammierung erlaubt die Automatisierung Fernwartung der Rechner übers Netz.
  - Linux und Windows stellen aber unterschiedliche Shells zur Verfügung. Also sollte man deren Möglichkeiten kontrastiv erkunden / lernen

- **bash / dash** = Teil des Betriebssystems Linux
- Linux bereitzustellen, impliziert, bash / dash bereitzustellen

Deshalb: **Varianten der Linux-Bereitstellung** 'neben' Windows:

3.1 Linux+bash *auf einem zweiten Rechner/Laptop*

- A) Ubuntu-24.04.03 LTS ISO-Datei von https://ubuntu.com/download/desktop herunterladen.
- B) Mit Windowsmitteln einen bootfähigen USB-Stick erzeugen.
- C) Rechner von USB-Stick booten.
- D) Installationsanleitung folgen.

3.2 Linux+bash *als Dual Boot Installation*

- A) Verfahren wie unter (3.1).
- B) Bei Installation 'neben Windows installieren' auswählen

3.3 Linux+bash *in einer virtuellen Maschine unter Windows*

- A) Virtualisierungssoftware auf Host *Windows* installieren
  - Empfohlen: →Virtual Box. Ist GPL v3 lizenziert = echte freie Software → https://www.virtualbox.org/wiki/Lice
  - Abgeraten: VMware Workstation Player (kurz VMplayer) weil
    * → Vmware gekauft von Broadcom.
    * Broadcom monetarisiert die vorher (für den privaten Gebrauch) "freie" Software.
    * Lizenzen für VMware Workstation sind über diese Produktnummer nicht mehr erhältlich. Die Nachfolge-Lizenz war VMware Desktop Hypervisor Pro, diese ist aber auch im November 2024 nicht mehr erhältlich → https://www.software-express.de/hersteller/vmware/workstation/player/
    * Antwort der Community: GitHub-Projekt zum Sammeln existierender Keys. → https://github.com/hegdepavankumar/VMware-Workstation-Pro-17-Licence-Keys.
    * Aber trotzdem keine seriöse Variante mehr.

- B) Ubuntu-24.04.03 LTS ISO-Datei von → https://ubuntu.com/download/desktop herunterladen.
- C) In Virtualisierungssoftware als Start-/Bootmedium aktivieren.

- D) Virtuelle Maschine booten.
- E) Installationsanleitung folgen

3.4 Linux+bash *in einer virtuellen Maschine unter MacOs*

- A) Virtualisierungssoftware auf Host *MacOs* installieren
  - mögliche: → Virtual Box. Ist GPL v3 lizenziert = echte freie Software → https://www.virtualbox.org/wiki/Licen
  - getestet: [→ UTM]{https://mac.getutm.app/}. Ist Apache-v2 lizenziert = echte freie Software → https://github.com/utmapp/UTM. Binaries auf Github und im AppleStore, dort aber angeblich gegen Geld.
- B) Ubuntu-24.04.03 LTS ISO-Datei von → https://ubuntu.com/download/desktop herunterladen.
- C) In Virtualisierungssoftware als Start-/Bootmedium aktivieren.
- D) Virtuelle Maschine booten.
- E) Installationsanleitung folgen

3.5 Linux+bash *Windows Subsystem Linux*

- A) Über Windowssuchzeile nach *Features* suchen lassen
- B) Button *Windows-Features aktivieren …* anklicken
- C) Im Dialog *Windows Subsystem für Linux* anwählen/aktivieren
- D) Neustart
- E) Powershell 7.5.x nach Windowsanleitung installieren → https://learn.microsoft.com/en-us/powershell/scripting/install/installing-powershell-on-windows?view=powershell-7.5
- F) Powershell (Adminmode) aufrufen
- G) gemäß → Microsoftanleitung WSL-Aktivierung verfahren - bzw.
  - a) powershell: `wsl --install`
  - b) powershell: `wsl --set-version UBUNTU`

4 Linux+bash *Verifikation*

4.1 Linux+bash *in virtueller Maschine oder als Dual-Boot-System*

- A) Linuxshell bzw. öffnen
- B) > `ping 8.8.8.8` eingeben
- Wenn Rückgabe *64 bytes from 8.8.8.8*, dann
  - Zugriff auf Linuxtool *ping* (Linux läuft)
  - und Netzzugriff auf Google-Server (Netzzugang per NAT aktiviert)

4.2 Linux+bash *in WSL auf Windows 11 Rechner*

- A) Powershell öffnen
- B) > `pwsh` eingeben (= Wechsel zu Powershell 7)
- C) > `ping 8.8.8.8` eingeben (= Netzzugang verifizieren)
- D) > `ipconfig` eingeben (= Windowsanalyse Netzwerkinterface)
- E) > `wsl` eingeben (= WSL aktivieren)
- F) > `ifconfig` eingeben (= Linuxanalyse Netzwerkinterface)

Hinweis: *Falls `ifconfig` oder gar `ping` unter LNX oder WSL fehlt, bitte in der Shell eingeben: `sudo apt-get install net-tools`

ÜBUNG **LF11a:sbj-00.sh2go:02**

- ☐ Installieren Sie eine Ihnen genehme Linux-Variante.
- ☐ Verifizieren Sie Ihre Installation

---

5. *VSCODE als Shellscript-Editor*

VSCODE

- wird von Microsoft zusammen mit der Community unter der MIT-Lizenz auf GitHub entwickelt [https://github.com/microsoft/vscode]https://github.com/microsoft/vscode()
- ist echte Open-Source-Software
- wird auch als Binärpaket für Windows, Linux und Mac bereitgestellt → https://code.visualstudio.com/download
- kann mit Extensionen funktionell erweitert werden.

Hinweise:

- Die Binärpakete sind mit einer gesonderten Microsoft-Lizenz verbunden → https://code.visualstudio.com/license. Die erlaubt Windows, ihre Binärpakete mit 3rd-Party-Komponenten anzureichern. Außerdem sind Extensionen u.U. anders lizenziert.
- Wer Extensionen aus dem Windowsmarketplace installiert, stimmt den → Visual-Studio-Marketplace-Terms-of-Usex. Die besagen:

> *"Marketplace Offerings are intended for use only with In-Scope Products and Services and you may not install, reverse-engineer, import or use Marketplace Offerings in products and services except for the In-Scope Products and Services."* S.2

Das bedeutet, dass Extensionen aus dem Windowsmarketplace nur in den von Microsoft bereitgestellten Binaries integriert werden dürfen.

Die Community stellt aber unter dem Namen → VSCODIUM eine Version ohne die Microsoft integrierten 3rd-Party-Komponenten. Deren Extension müssen konsequenterweise aus einem gesonderten Repository [→ https://open-vsx.org/ stammen.

Installation

- VSCODE (Visual Studio Code) für
  - → Windows (https://code.visualstudio.com/docs/setup/windows)
  - → Linux (https://code.visualstudio.com/docs/?dv=linux64_deb)
  - → `VSCODIUM` für
    - → Windows
    - → Linux
  - VS Plugins/Extensions jeweils über *Preferences/Extensions* installieren lassen. D.h. für
    - `VSCODE` erweitert aus → Visual Studio Marketplace:
      - PLUGINs für bash programming unter VSCODE: `shfmt`
      - PLUGINs für pwsh programming unter VSCODE: `pwsh`
    - `VSCODIUM` :
      - PLUGINs für bash programming unter VSCODIUM `shfmt`
      - PLUGINs für pwsh programming unter VSCODIUM: `pwsh`

Verifikation

ÜBUNG **LF11a:sbj-00.sh2go:03**

☐ Öffnen Sie das Script `sh2go-00-happy-coding.sh` in Ihrer VS-Umgebung.

- ☐ Ersetzen Sie ‘Happy Coding’ durch eine Botschaft Ihrer Wahl.
- ☐ Sichern Sie das geänderte Script unter dem Namen `sh2go-00-mymsg.sh`
- ☐ Öffnen Sie in Ihrer Linux-Umgebung eine Commandshell (bash).
- ☐ Lassen Sie das Script `sh2go-00-mymsg.sh` ausführen.

---

6. sh2go als (Selbst)Lernprogramm

6.1 Vorbereitung

ÜBUNG **LF11a:sbj-00.sh2go:04**

- ☐ Laden Sie von `https://github.com/protirone/protirone.lessons/` aus dem Ordner `fachinformatik/lf.cx` die Zip-Datei `cx.sh2go-snp.zip`
- ☐ Entpacken Sie die Zip-Datei unter Linux in einem Arbeitsordner Ihrer Wahl
- ☐ Laden Sie die Datei `sh2go-00-happy-coding.sh` in Ihren VSCODE-Editor und lassen Sie sie ausführen.

6.2 Einführung

- `bash` und `dash` haben wenig eingebaute Befehle.
- Die meisten Fähigkeiten (zur Manipulation von Dateinamen und -inhalte) habe Shellskripte, weil sie Betriebssystemkommandos aufrufen können:

```
1 #!/bin/bash
2
3 echo "Hello World"
4
5 # ruft das externe Linuxtool `echo` auf
```

- Ganz zuletzt sind `bash`- und `dash`-Skripte also geschickt arrangierte Aufrufe von Betriebssystembefehlen.
- Übersicht siehe `lnx-cmds.mm` = Freeplane-Datei = Mindmap der gebräuchlichsten Kommandozeilentools.
- Die, die ich davon täglich nutze sind:

  - `cp` (= *copy*)
  - `date`
  - `echo`
  - `find`
  - `grep`
  - `move`
  - `sed`
  - `sort`
  - `test`
  - `uniq`
  - `wc`

- Hinzu kommen die built-in-Kontrollkonstrukte **`if`**, **`while`**, **`for`** und die built-in-Zuweisungs- und Evaluationsoperatoren =, Backticks, `$`, `$arg` etc.

Der Ordner `snp.sh2go` bietet für diese Funktionen Aufgaben mit integrierten Anleitungen und die entsprechenden Lösungen. Beispiel:

- `sh2go-01-echo.sh` beschreibt die Aufgabe und enthält Hintergrundinformationen.
- `sh2go-01-echo.sol.sh` bietet darüber hinaus eine Lösung dafür an.

ÜBUNG **LF11a:sbj-00.sh2go:05**

sh2go ist gedacht als (Selbst)Lernprogramm für die Shellprogrammierung:

- ☐ Gehen Sie die Aufgaben sh2go-00* - sh2go-21 durch.
- ☐ Öffnen Sie die Aufgabendatei in einem Shellskript-Editor Ihrer Wahl.
- ☐ Lösen Sie die Aufgabe zu erst selbst. (Ihr Kern ist immer in einer Zeile umsetzbar)
- ☐ Testen Sie Ihren Ansatz durch das Aufrufen Ihrer Lösung in *bash* oder *dash*
- ☐ Vergleichen Sie Ihre Lösung mit der zugehörigen Lösungsdatei: sh2go-XY*.sol.sh

---

PS.: Parallele Aufgaben für die Powershell stehen noch aus.